

# Hermes vs TableAgent

## 0. 先给两个框架画个性格像

如果把 agent 框架比作一个办公室：

- **Hermes** 像一家已经制度化的智能助理公司：有前台入口、长期档案室、任务计划表、技能培训库、工具网关、隔离工位、跨会话搜索和后台学习闭环。它追求的是“什么事都能接，接完还能学习，下次更熟”。
- **TableAgent** 更像一个很懂 Excel 的专项分析组：办公室不大，但桌上有财务报表、工具箱、记忆笔记、夜间复盘员和表格老会计。它的强项不是泛用，而是围绕表格、schema、domain knowledge、sheet/header 识别做深。

所以 TableAgent 的升级重点不是变成 Hermes，而是学习 Hermes 的“管理方法”：

让记忆能检索，让技能能按条件触发，让工具有统一入口，让后台学习有明确流向，让每次表格答案能追溯。

## 1. 框架主线对比：Hermes 是“系统化智能体”，TableAgent 是“表格专项智能体”

### Hermes 的主线

Hermes 的公开文档里，核心模块大致包括：

- CLI / Gateway：多入口接入。
- Memory Provider：持久记忆与跨会话检索。
- Skills：可复用能力，支持命名空间、激活条件、依赖和多种后端。
- Tools / MCP：统一工具层。
- Subagents / isolated runtime：隔离执行子代理。
- Cron / scheduler：计划任务。
- Evaluations / trajectory compression：评测和轨迹压缩。

它像一个“全功能智能体操作系统”。

### TableAgent 的主线

从当前报告和代码路径看，TableAgent 的主线是：

- **ContextBuilder**：拼装系统上下文，包含身份、记忆、技能摘要、工具契约和近期历史。
- **MemoryStore / Consolidator / Dream**：文件型长期记忆、历史 JSONL、上下文压缩、后台反思整理。
- **SessionManager**：会话 JSONL 持久化。
- **ToolRegistry / ToolLoader**：内置工具、插件、MCP 工具、schema 和 scope。
- **TableClaw**：表格领域知识、schema cache、header 识别、样例抽样、月份/范围/主体推断。
- **SubagentManager**：后台子代理。

它像一个“表格分析流水线”：先装上下文，再找工具，再读表，再把结果和经验记下来。

### 最重要的判断

TableAgent 已经有 Hermes 的影子：记忆、技能、工具、子代理、Dream 后台整理都有。真正缺的不是“模块”，而是 **模块之间的统一协议和启发式路由**。

换句话说，TableAgent 不是没有器官，而是神经系统还不够清晰。

---

## 2. 把“最近历史注入”改成“轻量 session search”

Hermes 给的启发

Hermes 强调跨会话搜索和持久记忆，不只是把最近对话塞进 prompt。它的思想是：  
**过去的内容不是按时间排队进场，而是按相关性被召回。**

TableAgent 当前实现

TableAgent 当前的 `ContextBuilder` 会把长期记忆、会话摘要、尚未被 Dream 消化的近期历史等拼进上下文。报告里提到近期历史默认最多 50 条，还有字符上限。

这很实用，但对表格场景有一个典型问题：

用户今天问“继续上次那个长账龄口径，换成绵阳”，真正有价值的是上次“长账龄口径”的定义和用过的表，不一定是最近 50 条里的所有内容。

对表格业务的优势

这会让 TableAgent 从“记得刚才聊过什么”升级为“能找回和当前表格问题相关的旧经验”。

尤其适合这些场景：

- “还是刚才那个口径”
- “沿用上次的表”
- “按之前那个剔除合计行规则”
- “上次你说 ICT 欠费字段在哪一列”

现在像分析师翻聊天记录；升级后像资料员直接递来“长账龄口径”那张卡片。

---

## 3. 给 Skills 增加“激活条件”，而不是只靠描述文字

Hermes 给的启发

Hermes 的 Skills 体系比普通 prompt 更制度化。技能不是“全部塞进脑子”，而是有名称、命名空间、触发条件、依赖和运行后端。它强调的是：  
**技能要在合适的时候出现，不合适的时候安静。**

TableAgent 当前实现

TableAgent 已经有 `SkillsLoader`，并且升级报告提到它兼容 nanobot / openclaw 风格的 frontmatter。这是很好的地基。

当前可以进一步补的，是技能路由的颗粒度。表格场景里，技能触发不能只靠大模型读描述猜，因为有些技能非常像：

- “百分比归一化”
- “同比/环比计算”
- “合计行排除”
- “财务欠费口径”
- “多 sheet 表头定位”

如果都靠 prompt 自己想，容易该用时不用，不该用时乱用。

## 对表格业务的优势

表格 Agent 最怕“专业小坑”。比如：

- 表里写 0.128，回答时到底是 0.128、12.8%，还是 0.128%？
- 用户说“提高 2 个点”，不是“提高 2%”。
- 排名字段不能被当作金额字段求和。
- “合计”行不能和地市行一起聚合。

技能激活条件就像给分析师桌上贴便利贴：看到“百分点”就自动翻那本手册，看到“合计行”就自动检查过滤规则。

---

## 4. 把 TableClaw 的 schema cache 变成“可问的索引”

### Hermes 给的启发

Hermes 的记忆和会话搜索说明了一个原则：

**存下来不等于能用；能被检索、排序、解释，才是真正的记忆。**

### TableAgent 当前实现

tableclaw.py 已经有 schema cache、sheet/header 识别、文件新鲜度判断、样例行抽样、domain knowledge 匹配。这些能力非常接近“表格索引”的雏形。

但当前 cache 更像“下次少扫一遍表”的加速器，还没有完全变成“我能问它哪个表有这个指标”的检索层。

## 对表格业务的优势

这一步会直接改善“找表”和“找列”。

比如用户问：

5 月四川各市州长账龄欠费 top10 是什么？

现在 TableAgent 更可能先扫表、猜字段。

升级后可以先查：

```
FTS: 长账龄 欠费 四川 市州
Filter: month=202405
Return:
  file=xxx.xlsx
  sheet=欠费明细
```

candidate columns=地市，长账龄欠费，应收账款  
confidence=0.86

比喻一下：  
现在的 schema cache 像分析师在 Excel 封面贴了便签；升级后的 TableIndex 像把所有便签录进了检索系统。

## 工具结果统一加“来源凭证”

### Hermes 给的启发

Hermes 把 tools、MCP、skills、gateway 放在统一运行时里。对 TableAgent 来说，不必复制整个 gateway，但可以学习它的统一工具事件观：  
**工具调用不只是执行动作，还要留下可回放、可追踪、可学习的轨迹。**

### TableAgent 当前实现

当前 Tool 抽象已经有 JSON Schema、读写属性、scope、并发安全标志等。ToolRegistry 也会稳定排序工具 schema。

这说明工具层不是薄弱点。薄弱点是：表格工具的输出如果只是一段文本，后续很难判断：

- 答案来自哪个 workbook？
- 哪个 sheet？
- 哪个行列范围？
- 是否包含合计行？
- 是否可以写入长期记忆？
- 是否可作为评测回放依据？

### 对表格业务的优势

这能立刻提升可信度。表格问题不是只要答案，还要“凭什么”。

业务用户真正想听的是：

成都长账龄欠费是 123.45 万元，来自 202405\_欠费报表.xlsx / 地市明细 / 第 12 行 / 长账龄欠费  
列，已排除合计行。

这就像财务分析师不只报数字，还把凭证夹在报告后面。

## 7. 给表格流程加一个“轻量状态机”

### Hermes 给的启发

Hermes 有更完整的 agent runtime 和任务流思想。TableAgent 不必上完整状态图，但可以给表格分析加一个轻量状态机。

### TableAgent 当前实现

现在的流程更像：

用户问 -> 模型判断 -> 调工具 -> 读表 -> 回答

这很灵活，但表格业务其实有几个固定阶段：

识别问题

- > 定位文件/sheet
- > 对齐 schema
- > 抽取/计算
- > 校验单位和口径
- > 给出来源

这些阶段如果完全靠模型自由发挥，偶尔会跳步。

不用强制所有问题都走完整流程，但对“数值型问题”默认走。

## 对表格业务的优势

这会减少最常见的“跳步错误”：

- 没定位清楚 sheet 就开始回答。
- 没确认单位就输出数字。
- 没排除合计行就排序。
- 没给来源就给结论。

比喻一下：

现在像经验丰富的人一边翻表一边算；升级后像有一张财务审核清单，算完必须过几道章。